UNIVERSIDADE FEDERAL DE SERGIPE

Filas: conceito e implementação

Entra-se por uma ponta e sai-se pela outra

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje

- Fila: o primeiro a entrar é o primeiro a sair
- Fila num vetor: `ini` e `fim` só andam para a direita
- A fila circular: o fim dá a volta no vetor
- Quando o vetor circular enche: dobrar sem embaralhar
- Fila numa lista: sai do primeiro nó, entra depois do último
- Quanto custa: deslocar, dar a volta ou encadear
- Exercícios

A pergunta de hoje

O problema: quem fica com a batata?

Até aqui

A pilha devolve tudo na ordem *inversa*: o último a entrar sai primeiro. Serviu para parênteses, desfazer e a pilha de execução.

O que vamos descobrir

Muitos problemas querem a ordem de **chegada**: quem chegou antes é atendido antes. A estrutura é tão simples quanto a pilha — mas guardá-la num vetor esconde uma armadilha.

A pergunta de hoje

n crianças em roda, numeradas de 1 a *n*. A batata quente passa de mão em mão *k* vezes; quem fica com ela sai, e a brincadeira continua a partir do seguinte. Quem sobra?

O programa de hoje: batata.c (1/2)

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
typedef struct {
    int v[MAX]; /* frente: v[ini] */
    int ini, fim; /* entra em v[fim] */
} Fila;

void enfileira(Fila *f, int x)
{
    if (f->fim == MAX) {
        fprintf(stderr, "fila cheia\n");
        exit(1);
    }
    f->v[f->fim] = x;
    f->fim++;
}
```

```c
int desenfileira(Fila *f)
{
    if (f->ini == f->fim) {
        fprintf(stderr, "fila vazia\n");
        exit(1);
    }
    int x = f->v[f->ini];
    f->ini++;
    return x;
}

int tamanho(const Fila *f)
{
    return f->fim - f->ini;
}
```

batata.c (2/2): a roda e o main

```c
/* imprime quem sai; devolve quem fica */
int roda(int n, int k)
{
    Fila f = {.ini = 0, .fim = 0};
    for (int i = 1; i <= n; i++)
        enfileira(&f, i);
    while (tamanho(&f) > 1) {
        for (int j = 0; j < k; j++)
            /* passa a batata */
            enfileira(&f, desenfileira(&f));
        /* quem ficou com ela sai */
        printf(" %d", desenfileira(&f));
    }
    return desenfileira(&f);
}
```

```c
int main(void)
{
    int casos[][2] = {
        {6, 0}, {5, 2}, {7, 1},
    };
    for (int i = 0; i < 3; i++) {
        printf("n = %d, k = %d: sai",
               casos[i][0], casos[i][1]);
        int fica = roda(casos[i][0],
                        casos[i][1]);
        printf(" | fica %d\n", fica);
    }
    return 0;
}
```

O que este programa imprime?

Antes de compilar, escreva sua resposta

1. Para $k = 0$, a batata nem passa. Em que ordem as seis crianças saem?
2. Para $n = 5$ e $k = 2$, quem sai primeiro? Quem sobra?
3. E para $n = 7$ e $k = 1$?
4. Passar a batata é `enfileira(&f, desenfileira(&f))`. O que essa linha faz com a criança da frente?

Compile, execute e compare

```
$ gcc -Wall -Wextra batata.c -o batata
$ ./batata
n = 6, k = 0: sai 1 2 3 4 5 | fica 6
n = 5, k = 2: sai 3 1 5 2 | fica 4
n = 7, k = 1: sai 2 4 6 1 5 3 | fica 7
```

O que o programa guarda

A roda inteira, na ordem: quem está com a batata na **frente**, quem acabou de passá-la no **fim**.

Passar a batata tira a criança da frente e a põe no fim: a roda gira sem que ninguém mude de lugar em relação aos outros. Com $k = 0$, saem na ordem em que entraram.

Fonte: Graham, Knuth e Patashnik (1994), §1.3, *o problema de Josephus* [5]

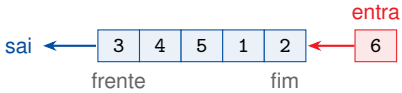
O que é uma fila

Definição: fila

Fila

Uma sequência em que as inserções acontecem numa ponta, o *fim*, e as remoções na outra, a *frente*. O primeiro a entrar é o primeiro a sair: *FIFO*, *first in, first out*.

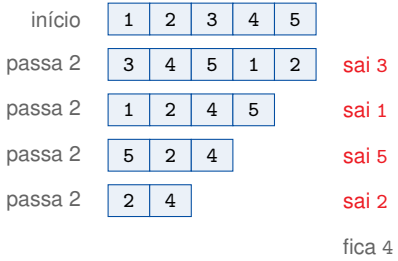
`enfileira(f, x)`	põe `x` no fim
`desenfileira(f)`	tira e devolve a frente
`frente(f)`	devolve a frente sem tirar
`vazia(f)`	a fila não tem nada?



Na pilha, as duas operações usam a mesma ponta. Na fila, cada uma usa a sua — é daí que vem toda a diferença na implementação.

Fonte: Cormen et al. (2009), §10.1 [3]; Celes et al. (2004), cap. 12 [2]

Seguindo a fila: $n = 5$, $k = 2$



Cada passada tira da frente e põe no fim; a criança que sai é a que está na frente depois das k passadas. Ninguém precisa ser procurado.

Fonte: Graham, Knuth e Patashnik (1994), §1.3 [5]; Tenenbaum et al. (1995), cap. 4 [12]

Filas que você já usou

O teclado

O que você digita espera num *buffer* até o `scanf` ler — na ordem em que foi digitado. É por isso que o `\n` esquecido aparece no próximo `scanf`.

A impressora

Os documentos esperam na fila de impressão; o primeiro mandado é o primeiro impresso.

O processador

O sistema dá a cada programa uma fatia de tempo e o põe no fim da fila de prontos — é a batata quente, sem ninguém sair.

Em todos, a pergunta é a mesma: *quem está esperando há mais tempo?*

Fonte: Tanenbaum e Bos (2015), §2.4.3, *round-robin* [11]; Kernighan e Ritchie (1988), §8.5 [7]

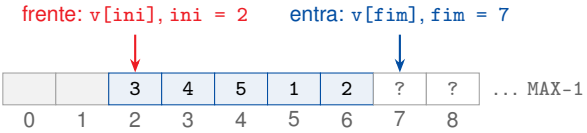
Recapitulando: o que é uma fila

O que aprendemos

- fila: entra no fim, sai da frente — o primeiro a entrar é o primeiro a sair;
- quatro operações: `enfileira`, `desenfileira`, `frente` e `vazia`;
- passar a batata é desenfileirar e enfileirar de novo: a fila “gira” sem mudar a ordem relativa;
- buffers, impressoras e o escalonador do sistema são filas.

Fila num vetor

A frente é `v[ini]`, o próximo entra em `v[fim]`



`enfileira`: escreve em `v[fim]`, depois `fim++`.
`desenfileira`: lê `v[ini]`, depois `ini++`.
`vazia`: `ini == fim`; `tamanho`: `fim - ini`.

Nenhuma operação tem laço. Mas repare: na pilha só o topo andava, e para os dois lados. Aqui `ini` e `fim` **só aumentam**. As posições à esquerda de `ini` nunca mais são usadas.

Fonte: Cormen et al. (2009), §10.1, ENQUEUE e DEQUEUE [3]; Celes et al. (2004), cap. 12 [2]

E se a roda for maior?

```
$ ./batata 41 2
```

O `main` completo aceita `n` e `k` pela linha de comando. São 41 crianças — a fila nunca tem mais do que 41 elementos — e `MAX` é 100.

Pense antes de passar o slide

1. Cabe? Algum dos dois testes de `enfileira` e `desenfileira` vai disparar?
2. Quantas vezes `enfileira` é chamada até sobrar uma criança?
3. Quanto vale `fim` no final?

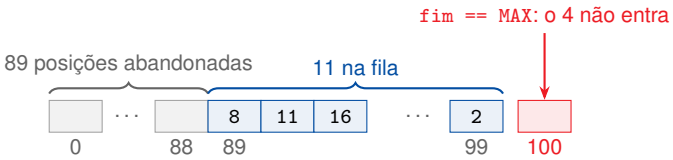
Execute e compare

```
$ ./batata 41 2
n = 41, k = 2: sai 3 6 9 12 15 18 21 24 27 30 33 36 39 1 5 10 14 19 23 28
                   32 37 41 7 13 20 26 34 40
enfileira: fila cheia
```

Saíram 29 crianças; restavam 12 na roda. A fila “encheu” com 12 elementos num vetor de 100.

Desta vez não houve lixo nem estouro silencioso: o teste de `enfileira` — a lição da aula de pilhas — parou o programa. Mas parou um programa que deveria funcionar.

A resposta: a fila anda para a direita



Cada criança que entra e cada passada da batata é um `enfileira`: são $n + k(n-1) = 41 + 2 \cdot 40 = 121$ ao todo, e `fim` só cresce. A fila nunca passou de 41 elementos, mas foi andando para a direita e bateu na parede, deixando para trás posições vazias que ninguém mais usa.

Um vetor de 100 só aguenta 100 `enfileira` **na vida inteira** da fila. Não basta saber quantos elementos cabem: é preciso reaproveitar o espaço que a frente deixou.

Fonte: Cormen et al. (2009), §10.1 [3]; Tenenbaum et al. (1995), cap. 4 [12]

Primeira ideia: trazer todo mundo para a frente

```c
int desenfileira(Fila *f)
{   /* a frente e sempre v[0] */
    int x = f->v[0];
    for (int i = 1; i < f->fim; i++)
        f->v[i - 1] = f->v[i];
    f->fim--;
    return x;
}
```

Sem `ini`: quem sai é sempre `v[0]`, e os outros andam uma casa para a esquerda. Agora `MAX` = 41 basta para a roda de 41.

É assim numa fila de banco: quando o primeiro é atendido, todos dão um passo. Mas no vetor cada passo é uma cópia — com n na fila, **cada** `desenfileira` custa $n - 1$ cópias.

Fonte: Celes et al. (2004), cap. 12 [2]; medido com `custo.c`, adiante

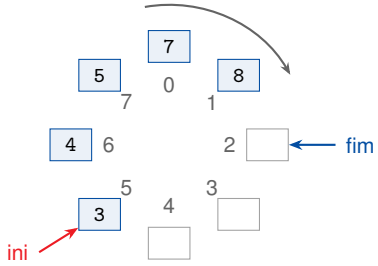
Recapitulando: fila num vetor

O que aprendemos

- com `ini` e `fim`, enfileirar e desenfileirar não têm laço: escrevem em `v[fim]` e leem de `v[ini]`;
- mas os dois índices só crescem: a fila anda pelo vetor e "enche" depois de `MAX` enfileiramentos, não com `MAX` elementos;
- deslocar todos a cada saída resolve o espaço, mas faz cada `desenfileira` custar o tamanho da fila;
- falta reaproveitar o espaço que a frente deixa para trás.

A fila circular

O fim dá a volta



Imagine o vetor dobrado em anel: depois de `v[MAX-1]` vem `v[0]`. Quando o fim chega à última posição, continua no começo — no espaço que a frente já liberou.

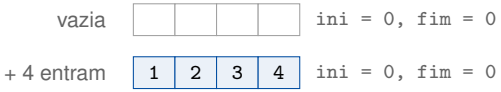
Em C, “dar a volta” é o resto da divisão:

```
fim = (fim + 1) % MAX;
ini = (ini + 1) % MAX;
```

Na figura, `MAX = 8`: a fila é `3 4 5 7 8`, com a frente em `v[5]` e o próximo entrando em `v[2]`.

Fonte: Cormen et al. (2009), §10.1, Figura 10.2 [3]; Knuth (1997), §2.2.2 [8]

Cheia ou vazia?



`MAX = 4`. Depois do quarto `enfileira`, o `fim` deu a volta: `(3 + 1) % 4 == 0`.

Pense antes de passar o slide

1. Com `ini` e `fim` apenas, como `vazia` distingue as duas situações?
2. `tamanho` pode continuar sendo `fim - ini`?
3. O que mais a fila precisaria guardar?

A resposta: guarde quantos há

Com `ini` e `fim`, a fila cheia e a vazia são o mesmo estado: `ini == fim`. E `fim - ini` pode ser negativo quando o fim já deu a volta.

Há duas saídas clássicas:

- guardar `n`, o número de elementos: vazia é `n == 0`, cheia é `n == MAX`, e o `fim` nem precisa ser guardado;
- deixar sempre uma posição vazia: cheia é `(fim + 1) % MAX == ini`.

Com `ini` e `n`

frente	`v[ini]`
próximo	`v[(ini + n) % MAX]`
vazia	`n == 0`
cheia	`n == MAX`

O livro de Cormen usa a segunda saída e guarda no máximo $\texttt{MAX} - 1$ elementos. Aqui usaremos a primeira.

Fonte: Cormen et al. (2009), §10.1 [3]; Knuth (1997), §2.2.2 [8]

A roda com a fila circular: batata_circular.c

```c
#define MAX 41

typedef struct {
    int v[MAX];
    int ini, n; /* frente, quantos */
} Fila;

void enfileira(Fila *f, int x)
{
    if (f->n == MAX) { /* cheia */ }
    f->v[(f->ini + f->n) % MAX] = x;
    f->n++;
}
```

```c
int desenfileira(Fila *f)
{
    if (f->n == 0) { /* vazia */ }
    int x = f->v[f->ini];
    f->ini = (f->ini + 1) % MAX;
    f->n--;
    return x;
}
```

`roda` e `main` não mudaram uma linha: só a fila por dentro. Os testes de cheia e vazia são os de `batata.c`.

Execute e compare

```
$ gcc -Wall -Wextra batata_circular.c -o batata_circular
$ ./batata_circular 41 2
n = 41, k = 2: sai 3 6 9 12 15 18 21 24 27 30 33 36 39 1 5 10 14 19 23 28
                   32 37 41 7 13 20 26 34 40 8 17 29 38 11 25 2 22 4 35 16 | fica 31
```

Os 121 `enfileira` cabem num vetor de **41**: o espaço que a frente libera é reaproveitado pelo fim.

Conta a lenda que Josephus e um amigo, numa roda de 41 em que cada terceiro saía, escolheram os lugares 16 e 31 — os dois últimos.

Fonte: Graham, Knuth e Patashnik (1994), §1.3 [5]

Recapitulando: a fila circular

O que aprendemos

- o vetor vira anel com `% MAX`: o fim continua no espaço que a frente liberou;
- com só `ini` e `fim`, cheia e vazia são o mesmo estado — guarde `n` (ou deixe uma posição sempre livre);
- a próxima posição livre é `(ini + n) % MAX`;
- agora `MAX` limita quantos estão na fila, não quantos já passaram por ela.

Quando o vetor circular enche

Dobrar, como na pilha: `dobra_errada.c`

```c
void enfileira(Fila *f, int x)
{
    if (f->n == f->cap) { /* dobra */
        f->cap *= 2;
        f->v = realloc(f->v,
                 f->cap * sizeof *f->v);
    }
    f->v[(f->ini + f->n) % f->cap] = x;
    f->n++;
}
```

Na pilha, crescer era um `realloc`. O `main` começa com `cap = 4`, enfileira de 1 a 4, desenfileira dois, enfileira 5, 6 e 7 — o 7 faz dobrar — e imprime tudo o que sair.

Pense antes de passar o slide

1. Desenhe o vetor de 4 antes do 7: onde estão `3 4 5 6` e quanto vale `ini`?
2. Desenhe o vetor de 8 depois do `realloc`. Onde o 7 é escrito?
3. Em que ordem os cinco saem?

Fonte: ISO/IEC 9899:2018, §7.22.3.5 [6]

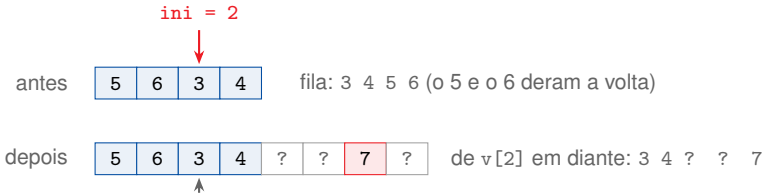
Execute e compare

```
$ ./dobra_errada
cap = 8, ini = 2, n = 5
sai: 3 4 0 0 7
$ gcc -g -fsanitize=address dobra_errada.c -o dobra_errada-asan
$ ./dobra_errada-asan
cap = 8, ini = 2, n = 5
sai: 3 4 -1094795586 -1094795586 7
```

O 5 e o 6 sumiram; no lugar deles saiu o que havia na memória nova — zero numa execução, outro valor na outra.

O *sanitizer* não acusa nada: todas as posições lidas estão *dentro* do bloco. Ele só preenche a memória nova com um padrão, `0xBEBEBEBE`, e o padrão aparece na saída.

A resposta: o `realloc` copia o vetor, não a fila



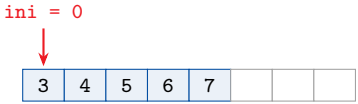
O `realloc` copia as posições 0 a 3 para as posições 0 a 3 do bloco novo. Só que, na fila, o 5 e o 6 vinham *depois* do 4: com `cap = 8`, a posição seguinte a `v[3]` deixou de ser `v[0]` e passou a ser `v[4]`, que é lixo.

A conta `(ini + i) % cap` mudou de significado quando `cap` mudou. Para crescer, é preciso copiar **na ordem da fila**, não na ordem do vetor.

Fonte: ISO/IEC 9899:2018, §7.22.3.5 [6]; Cormen et al. (2009), §17.4 [3]

Crescer desenrolando o anel

```c
/* fila_vetor.c: vetor com o dobro do
   tamanho, ja na ordem: frente em 0 */
static void cresce(Fila *f)
{
    int *novo = aloca(NULL,
                      2 * f->cap * sizeof *novo);
    for (int i = 0; i < f->n; i++)
        novo[i] = f->v[(f->ini + i) % f->cap];
    free(f->v);
    f->v = novo;
    f->ini = 0;
    f->cap *= 2;
}
```



O laço percorre a fila da frente para o fim, com o `cap` *antigo*, e escreve a partir de `novo[0]`.

Ainda são n cópias por crescimento, como na pilha: dobrando, o custo por `enfileira` continua constante, amortizado.

Fonte: Cormen et al. (2009), §17.4, *dynamic tables* [3]

Recapitulando: quando o vetor circular enche

O que aprendemos

- numa fila que deu a volta, a ordem dos elementos não é a ordem das posições no vetor;
- o `realloc` preserva as posições, e a conta `% cap` muda com o novo `cap`: a fila embaralha;
- cresça copiando na ordem da fila, com `ini` voltando a 0;
- dobrando, o custo amortizado de `enfileira` continua constante — a mesma conta da pilha.

Fila numa lista

Sai do primeiro nó, entra depois do último: `fila_lista.c`

```c
struct fila { No *ini, *fim; };

void enfileira(Fila *f, int x)
{   /* insere no fim */
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) exit(1);
    novo->valor = x; novo->prox = NULL;
    if (f->fim == NULL) f->ini = novo;
    else f->fim->prox = novo;
    f->fim = novo;
}
```

```c
int desenfileira(Fila *f)
{   /* remove o primeiro */
    if (f->ini == NULL) {
        fprintf(stderr, "fila vazia\n");
        exit(1);
    }
    No *velho = f->ini;
    int x = velho->valor;
    f->ini = velho->prox;
    if (f->ini == NULL) f->fim = NULL;
    free(velho);
    return x;
}
```



Fonte: Celes et al. (2004), cap. 12 [2]; Feofiloff (2009), cap. 5 [4]

Por que sai do início e entra no fim

A fila precisa de uma ponta para entrar e outra para sair. Na lista simples com `ini` e `fim`, três das quatro operações nas pontas são baratas:

	início	fim
inserir	barato	barato com `fim`
remover	barato	*n* passos

Remover o último exige achar o penúltimo — percorrer a lista toda. Então a remoção fica no início (a frente) e a inserção, no fim. Com isso, a lista simples basta; não é preciso `ant`.

Sem *overflow*

Como na pilha em lista, a fila só enche quando acaba a memória; o *underflow* continua possível.

Fonte: Celes et al. (2004), cap. 12 [2]; Cormen et al. (2009), §10.2, exercício 10.2-3 [3]

E se esquecer o `fim`?

```c
int desenfileira(Fila *f) /* esvazia.c */
{
    No *velho = f->ini;
    int x = velho->valor;
    f->ini = velho->prox;
    /* sumiu: if (f->ini == NULL) f->fim = NULL; */
    free(velho);
    return x;
}
/* main: */
enfileira(&f, 1);   enfileira(&f, 2);
printf("sai %d\n", desenfileira(&f));
printf("sai %d\n", desenfileira(&f));
enfileira(&f, 3);
printf("vazia? %s\n", vazia(&f) ? "sim" : "nao");
```

Pense antes de passar o slide

`enfileira` é a de `fila_lista.c`.

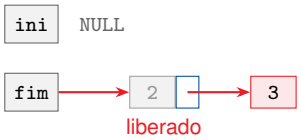
1. Depois dos dois `desenfileira`, quanto valem `ini` e `fim`?
2. Ao enfileirar o 3, qual ramo do `if` é seguido?
3. O que o programa imprime na última linha?

Execute e compare

```
$ ./esvazia
sai 1
sai 2
vazia? sim
$ gcc -g -fsanitize=address esvazia.c -o esvazia-asan && ./esvazia-asan
sai 1
sai 2
==96228==ERROR: AddressSanitizer: heap-use-after-free on address 0x602000000118
WRITE of size 8 at 0x602000000118 thread T0
    #0 0x000104bc5934 in enfileira esvazia.c:23
0x602000000118 is located 8 bytes inside of 16-byte region [0x602000000110,0x602000000120
    )
freed by thread T0 here:
    #1 0x000104bc5a8d in desenfileira esvazia.c:33
```

A linha 23 é o `f->fim->prox = novo;` de `enfileira`; a 33, o `free(velho);`. Relatório resumido.

A resposta: `fim` aponta para um nó que já foi liberado



Ao sair o 2, `ini` virou `NULL`, mas `fim` continuou apontando para o nó do 2 — já devolvido pelo `free`. Ao enfileirar o 3, `fim != NULL`: o programa escreve `fim->prox` em memória liberada e deixa `ini` em `NULL`.

O 3 foi alocado e "enfileirado", mas ninguém chega até ele: a fila se diz vazia e o nó vaza. A remoção do **último** é o caso especial da fila em lista: `ini` e `fim` mudam juntos.

Fonte: Celes et al. (2004), cap. 12 [2]; Serebryany et al. (2012) [10]

Uma interface, duas implementações

```c
/* fila.h */
#ifndef FILA_H
#define FILA_H

typedef struct fila Fila; /* campos no .c */

Fila *fila_cria(void);
void fila_libera(Fila *f);

void enfileira(Fila *f, int x);
int desenfileira(Fila *f);
int frente(const Fila *f);
int vazia(const Fila *f);
int tamanho(const Fila *f);

#endif
```

`fila_vetor.c` é a circular que cresce; `fila_lista.c`, a lista com `ini` e `fim`. `batata_tad.c` tem a mesma `roda`, agora com `Fila *`, e roda também $n = 100\,000$ com $k = 1$.

Pense antes de passar o slide

1. O que muda em `batata_tad.c` para trocar o vetor pela lista?
2. As duas versões dão o mesmo sobrevivente para $n = 100\,000$?

Fonte: Kernighan e Ritchie (1988), §4.5 [7]; Celes et al. (2004), cap. 9 [2]

Execute e compare

```
$ gcc -Wall -Wextra batata_tad.c fila_vetor.c -o batata_vetor
$ gcc -Wall -Wextra batata_tad.c fila_lista.c -o batata_lista
$ ./batata_vetor                          $ ./batata_lista
n = 6, k = 0: sai 1 2 3 4 5 | fica 6      n = 6, k = 0: sai 1 2 3 4 5 | fica 6
n = 5, k = 2: sai 3 1 5 2 | fica 4        n = 5, k = 2: sai 3 1 5 2 | fica 4
n = 7, k = 1: sai 2 4 6 1 5 3 | fica 7    n = 7, k = 1: sai 2 4 6 1 5 3 | fica 7
n = 41, k = 2: fica 31                    n = 41, k = 2: fica 31
n = 100000, k = 1: fica 68929             n = 100000, k = 1: fica 68929
```

`batata_tad.c` não mudou **nenhuma** linha: só o arquivo passado ao `gcc`. Nenhuma das duas versões tem `MAX`.

Para $k = 1$ há fórmula: escreva $n = 2^m + l$, com $0 \leq l < 2^m$; sobra $2l + 1$. Para $100\,000 = 2^{16} + 34\,464$, sobra $68\,929$.

Fonte: Graham, Knuth e Patashnik (1994), §1.3, equação (1.9) [5]

Recapitulando: fila numa lista

O que aprendemos

- a frente é o primeiro nó, o fim é o último: enfileirar é inserir depois de `fim`, desenfileirar é remover o primeiro;
- ao contrário da pilha, a fila precisa das **duas** pontas — e de `fim` para não percorrer a lista;
- quando sai o último, `fim` também vira `NULL`; esquecer isso deixa um ponteiro para memória liberada;
- com `fila.h`, quem usa a fila não sabe se ela é vetor ou lista.

Quanto custa

A roda com n crianças e $k = 1$: três filas

n	desloca	circular	lista
25 000	0,053 s	0,0004 s	0,001 s
50 000	0,239 s	0,0006 s	0,002 s
100 000	1,100 s	0,0013 s	0,005 s
200 000	4,217 s	0,0026 s	0,009 s

`custo.c`, `-O2`; as três dão o mesmo sobrevivente

Deslocando, dobrar n **quadruplica** o tempo: cada saída copia a fila inteira, e são n saídas.

Na circular e na lista, dobrar n dobra o tempo: custo fixo por operação. A lista é umas 4 vezes mais lenta que a circular — um `malloc` e um `free` por passada da batata.

Fonte: medido com `custo.c`; Bryant e O'Hallaron (2016), §6.2 e §9.9 [1]

Vetor ou lista?

	vetor que desloca	circular que dobra	lista com `fim`
`enfileira`	constante	constante (amortizado)	constante + `malloc`
`desenfileira`	**tamanho da fila**	constante	constante + `free`
pode encher?	sim, em `MAX`	só sem memória	só sem memória
caso delicado	—	crescer na ordem	`fim` ao esvaziar

Como na pilha, o vetor — aqui, circular — costuma ganhar. A lista compensa quando cada elemento é grande e já mora num nó, ou quando um `enfileira` nunca pode demorar mais que os outros.

Fonte: Cormen et al. (2009), §10.1 e §17.4 [3]; Celes et al. (2004), cap. 12 [2]

Recapitulando: quanto custa

O que aprendemos

- deslocar todo mundo a cada saída custa o tamanho da fila: o total cresce com n^2;
- na fila circular e na lista, toda operação tem custo fixo;
- a circular é a mais rápida na prática: memória contígua e crescimentos raros;
- a lista paga um `malloc` por `enfileira` e 16 bytes por `int`.

Mãos à obra

Altere o programa

Tarefa 1

Em `batata.c`, qual é o menor `MAX` com que `./batata 41 2` termina? Calcule antes, depois confira. E para `./batata 41 0`?

Tarefa 2

Faça a batata passar um número **diferente** de vezes a cada rodada: na rodada r (a partir de 1), ela passa r vezes. Preveja a ordem de saída para $n = 5$ antes de executar.

Tarefa 3

Em `batata_circular.c`, troque `MAX` por 40 e rode `./batata_circular 41 2`. Em que momento o programa para? Por que não para antes?

Exercícios (1/3)

Exercício 1 — uma fila que avisa

Na fila circular, `enfileira` devolve 0 se está cheia e `desenfileira(f, &x)`, 0 se está vazia. Na falha, a fila não muda.

Exercício 2 — inverter uma fila

Com uma pilha auxiliar, inverta a ordem de uma fila usando só as operações de `fila.h` e `pilha.h`.

Exercício 3 — escalonador *round-robin*

O processo i precisa de t_i unidades de tempo. O da frente recebe q unidades e, se não terminou, volta para o fim. Em que ordem e em que instante cada um termina?

Fonte: Celes et al. (2004), cap. 12 [2]; Tanenbaum e Bos (2015), §2.4.3 [11]

Exercícios (2/3)

Exercício 4 — fila com duas pilhas

Implemente `fila.h` usando só duas pilhas: uma recebe os que entram, a outra entrega os que saem. Quando passar de uma para a outra? Por que o custo amortizado é constante?

Exercício 5 — as últimas k linhas

Imprima as últimas k linhas da entrada, como `tail -n k`, com um vetor circular de k cadeias.

Exercício 6 — fila de duas pontas

Um *deque* insere e remove nas duas pontas. Implemente-o num vetor circular. Como andar para trás, se em C `-1 % cap` é -1?

Fonte: Cormen et al. (2009), §10.1, exercícios 10.1-5 e 10.1-6 [3]; Knuth (1997), §2.2.1 [8]

Exercícios (3/3)

Exercício 7 — Josephus sem simulação

Compare a simulação com a recorrência $J(1) = 0$, $J(n) = (J(n-1) + k + 1) \bmod n$ (posições a partir de 0), escrita recursivamente e com um laço. Até que n cada uma vai?

Exercício 8 — média móvel

Leia uma sequência de medições e imprima, a cada uma, a média das últimas k, em tempo constante por medição: guarde as k num vetor circular e mantenha a soma.

Exercício 9 — o caixa do banco

Clientes chegam em instantes dados, cada um com um tempo de atendimento. Com um caixa, calcule a espera de cada cliente e a média. E com dois caixas e uma fila única?

Fonte: Graham, Knuth e Patashnik (1994), §1.3 [5]; Tenenbaum et al. (1995), cap. 4 [12]

Desafio

O caminho mais curto no labirinto

Leia um labirinto (`#` parede, `.` livre, `S` saída, `E` entrada) e imprima o menor número de passos de `E` até `S`, andando nas quatro direções:

- enfileire `E` com distância 0;
- tire uma casa da frente e enfileire cada vizinha livre ainda não visitada, com distância + 1;
- a primeira vez que `S` sair da fila, a distância dela é a resposta — ou `sem caminho`.

```
#########
#E..#...#
#.#.#.#.#
#.#...#S#
#########
```

```
passos: 12
```

Por que a fila garante o *menor* caminho — e uma pilha, não?

Fonte: Moore (1959) [9]; Feofiloff (2009), cap. 5 [4]; Cormen et al. (2009), §22.2 [3]

Fechamento

O que mudou hoje

Usar as duas pontas, em vez de uma, fez o vetor “andar”. O anel e o `% cap` reaproveitam o espaço; a lista precisa de `ini` e de `fim`.

O que ficou

Pilha e fila escondem os campos atrás de um `.h` e oferecem só operações. Esse é o desenho de um *tipo abstrato de dados* — o tema da próxima aula.

Para levar

- fila: entra no fim, sai da frente — o primeiro a entrar sai primeiro;
- num vetor: anel com `% cap` e um contador `n`;
- para crescer: copie na ordem da fila, não na do vetor;
- numa lista: sai do primeiro nó, entra depois do último; ao esvaziar, `fim = NULL`.

Próxima aula: tipos abstratos de dados — implementação com structs e ponteiros.

Referências I


Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2016.
Seções 6.2 (localidade) e 9.9 (alocação dinâmica).


Waldemar Celes, Renato Cerqueira, and José Lucas Rangel.
Introdução a estruturas de dados: com técnicas de programação em C.
Elsevier, Rio de Janeiro, 2004.
Capítulos 9 (tipos abstratos de dados) e 12 (filas).

Referências II



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, Cambridge, MA, 3 edition, 2009.
Seções 10.1 (pilhas e filas), 10.2 (listas encadeadas), 17.4 (tabelas dinâmicas) e 22.2 (busca em largura).



Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 5: filas (fila circular e distâncias numa rede).

Referências III


Ronald L. Graham, Donald E. Knuth, and Oren Patashnik.
Concrete Mathematics: A Foundation for Computer Science.
Addison-Wesley, Reading, MA, 2 edition, 1994.
Seção 1.3: o problema de Josephus.


ISO/IEC.
ISO/IEC 9899:2018 — Information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seção 7.22.3.5 (`realloc`).

Referências IV



Brian W. Kernighan and Dennis M. Ritchie.
The C Programming Language.
Prentice Hall, Englewood Cliffs, NJ, 2 edition, 1988.
Seções 4.5 (arquivos de cabeçalho) e 8.5 (`getc` com *buffer*).



Donald E. Knuth.
The Art of Computer Programming, Volume 1: Fundamental Algorithms.
Addison-Wesley, Reading, MA, 3 edition, 1997.
Seções 2.2.1 (pilhas, filas e deques) e 2.2.2 (alocação sequencial e fila circular).

Referências V


Edward F. Moore.
The shortest path through a maze.
In *Proceedings of the International Symposium on the Theory of Switching*, pages 285–292, Cambridge, MA, 1959. Harvard University Press.
Origem da busca em largura com fila.


Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.

Referências VI


Andrew S. Tanenbaum and Herbert Bos.
Modern Operating Systems.
Pearson, Boston, 4 edition, 2015.
Seção 2.4.3: escalonamento *round-robin*.


Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
Capítulo 4: filas e listas.